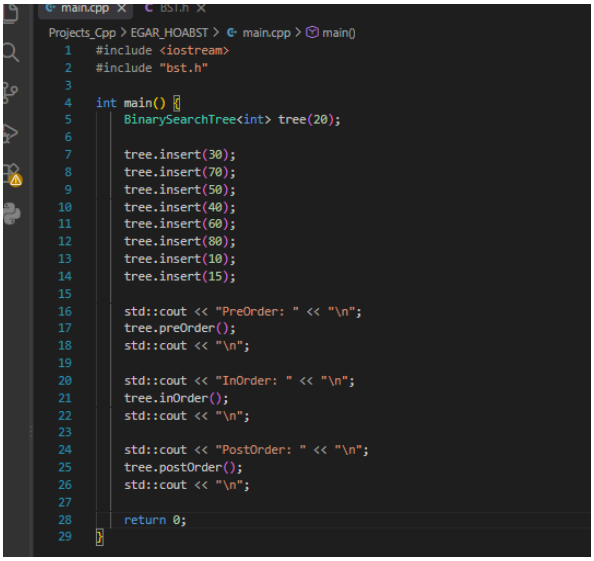
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	09/24/2026	**DATE SUBMITTED:**	09/24/2026
NAME:	Dela Providencia, Dane Diez, Angela May Doloso, Andrew Duenas, Ralph Egar, Jarel Dave	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 9.1 Tree ADT

1. PROCEDURE OUTPUT

Screenshot | Main.cpp	

Screenshot | BST.h

main.cpp BST.h

Projects_Cpp > EGAR_HOABST > BST.h > ...

```h
#ifndef BST_H
#define BST_H

#include <iostream>

template <typename T>
class BinarySearchTree {
private:
    // Value stored in the current node
    T key;

    // Pointers to the left and right child subtrees
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

public:
    // Constructor
    BinarySearchTree(T value);

    // Destructor
    ~BinarySearchTree();

    // Inserts a new value into BST
    void insert(T value);

    // Traversal Methods:
    // Root -> Left -> Right
    void preOrder();

    // Left -> Root -> Right
    void inOrder();

    // Left -> Right -> Root
    void postOrder();
};

// Constructor Implementation
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value) {
    key = value;
    left = nullptr;
    right = nullptr;
}
```

```h
// Destructor
template <typename T>
BinarySearchTree<T>::~BinarySearchTree() {
    delete left;
    delete right;
}

// Insertion
template <typename T>
void BinarySearchTree<T>::insert(T value) {
    // If the value is smaller, place it into the left subtree
    if (value < key) {
        if (left == nullptr) {
            left = new BinarySearchTree<T>(value);
        } else {
            left->insert(value);
        }
    }
    // If the value is larger, place it into the right subtree
    else if (value > key) {
        if (right == nullptr) {
            right = new BinarySearchTree<T>(value);
        } else {
            right->insert(value);
        }
    }

}
```

```
#include "BST.h"

// Pre order
template <typename T>
void BinarySearchTree<T>::preOrder() {
    // Process current node first
    std::cout << key << " ";

    // Traverse left subtree
    if (left != nullptr) {
        left->preOrder();
    }

    // Traverse right subtree
    if (right != nullptr) {
        right->preOrder();
    }
}

// In order
template <typename T>
void BinarySearchTree<T>::inOrder() {
    // Traverse left subtree
    if (left != nullptr) {
        left->inOrder();
    }

    // Process current node
    std::cout << key << " ";

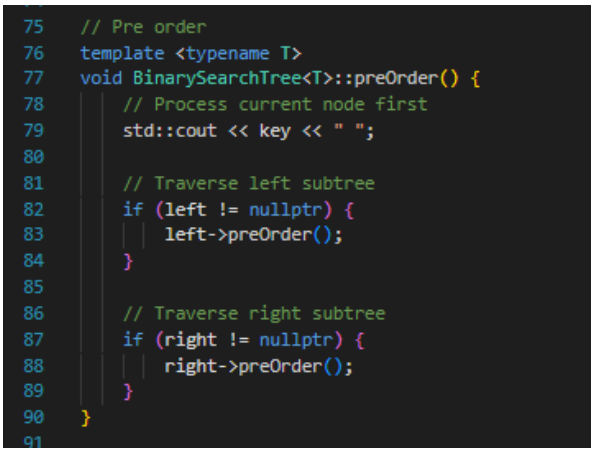
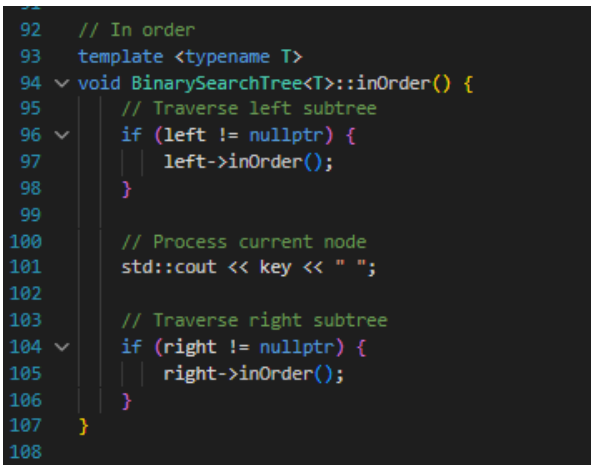
    // Traverse right subtree
    if (right != nullptr) {
        right->inOrder();
    }
}

// Post-Order Traversal: Left -> Right -> Root
template <typename T>
void BinarySearchTree<T>::postOrder() {
    // Traverse left subtree
    if (left != nullptr) {
        left->postOrder();
    }

    // Traverse right subtree
    if (right != nullptr) {
        right->postOrder();
    }


    std::cout << key << " ";
}

#endif
```

Screenshot | Preorder	
Screenshot | Inorder	
Screenshot | Postorder	

```cpp
// Pre order
template <typename T>
void BinarySearchTree<T>::preOrder() {
    // Process current node first
    std::cout << key << " ";

    // Traverse left subtree
    if (left != nullptr) {
        left->preOrder();
    }

    // Traverse right subtree
    if (right != nullptr) {
        right->preOrder();
    }
}
```

```cpp
// In order
template <typename T>
void BinarySearchTree<T>::inOrder() {
    // Traverse left subtree
    if (left != nullptr) {
        left->inOrder();
    }

    // Process current node
    std::cout << key << " ";

    // Traverse right subtree
    if (right != nullptr) {
        right->inOrder();
    }
}
```

```cpp
// Post order
template <typename T>
void BinarySearchTree<T>::postOrder() {
    // Traverse left subtree
    if (left != nullptr) {
        left->postOrder();
    }

    // Traverse right subtree
    if (right != nullptr) {
        right->postOrder();
    }


    std::cout << key << " ";
}

#endif
```

Output | Preorder, Inorder, Postorder	

CODE ANALYSIS: In this activity, we created a custom Binary Search Tree (BST). We organized the code using a header file (BST.h) and a main file (main.cpp) to keep it clean and organized. In the header file, we built a BinarySearchTree class that can work with any data type. We added an insertion function to organize the smaller numbers to the left and larger numbers to the right. We also added three traversal functions, which are the preOrder, inOrder, and postOrder. In main.cpp, we then built a tree starting with 20 as the root and inserted the values 30, 70, 50, 40, 60, 80, 10, and 15. Lastly, we then called all of the three traversal methods to print the values of the nodes. In conclusion, we learned how efficient Binary Search Trees (BST) are and how they work using diagrams.

Supplementary Activity

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

ROW 3 > main_supple (1).cpp > main()

```cpp
#include <iostream>
#include "BST_SUPPLE.h"

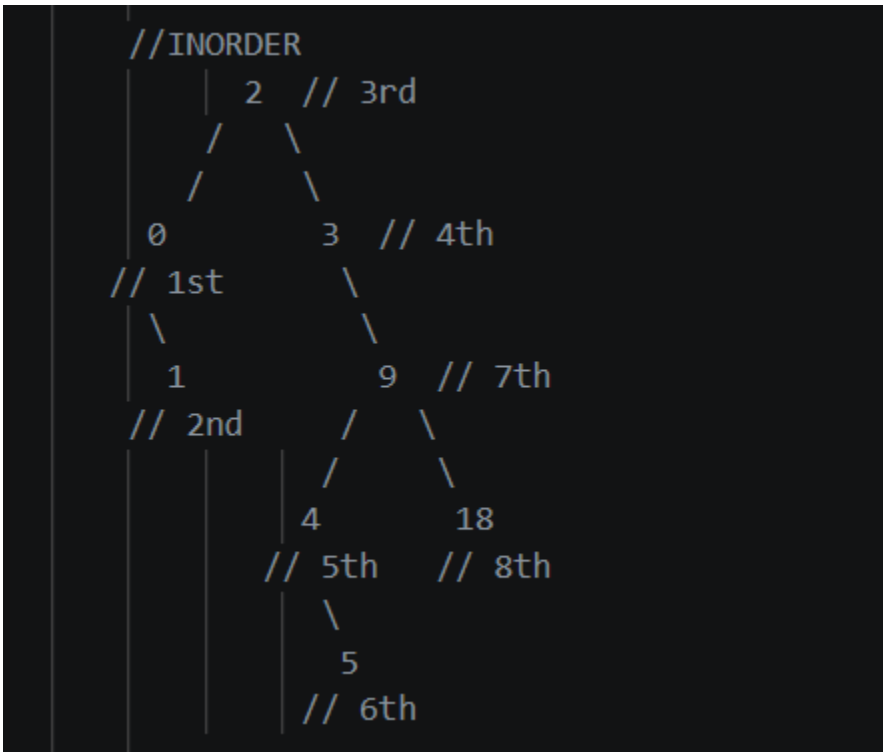
int main() {
    BinarySearchTree<int> tree(2);

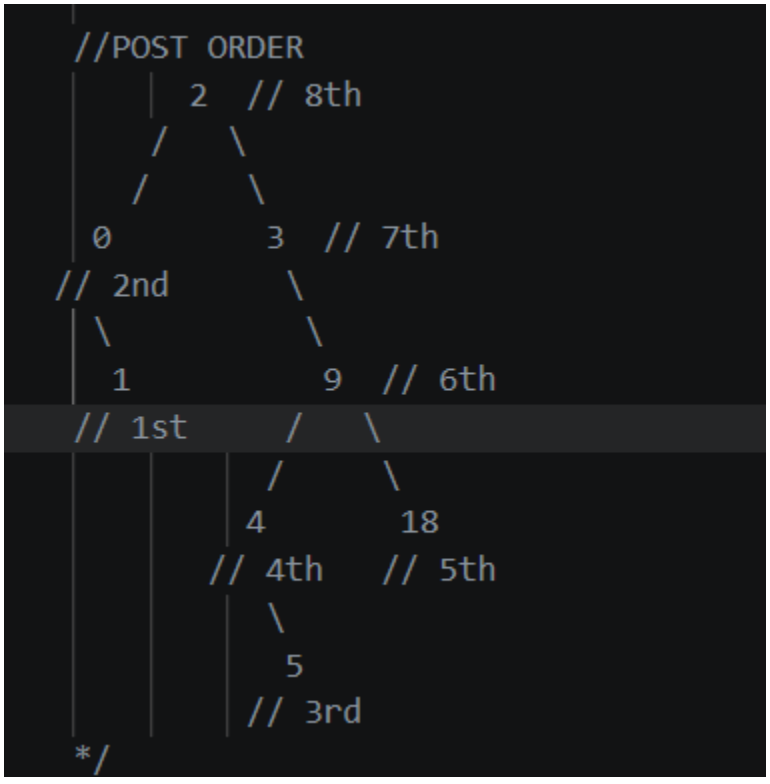
    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

```
14
15      // Tree structure:
16      /*
17                  2
18                 / \
19                0   3
20                 \   \
21                  1   9
22                     / \
23                    4   18
24                     \
25                      5
26      */
```

```
    //PREORDER
        2  // 1st
       /   \
      /     \
     0       3  // 4th
// 2nd       \
    \         \
     1         9  // 5th
// 3rd       /   \
            /     \
           4       18
        // 6th   // 8th
           \
            5
          // 7th
```


//INORDER
2 // 3rd
0 3 // 4th
// 1st
1 9 // 7th
// 2nd
4 18
// 5th // 8th
5
// 6th


//POST ORDER
2 // 8th
0 3 // 7th
// 2nd
1 9 // 6th
// 1st
4 18
// 4th // 5th
5
// 3rd
*/

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```
// In order
template <typename T>
void BinarySearchTree<T>::inOrder() {
    // Traverse left subtree
    if (left != nullptr) {
        left->inOrder();
    }

    // Process current node
    std::cout << key << " ";

    // Traverse right subtree
    if (right != nullptr) {
        right->inOrder();
    }
}
```

```
InOrder:
0 1 2 3 4 5 9 18
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

```cpp
// Pre order
template <typename T>
void BinarySearchTree<T>::preOrder() {
    // Process current node first
    std::cout << key << " ";

    // Traverse left subtree
    if (left != nullptr) {
        left->preOrder();
    }

    // Traverse right subtree
    if (right != nullptr) {
        right->preOrder();
    }
}
```

```
PreOrder:
2 0 1 3 9 4 5 18
```

Observation: The output starts with the root (2) and then explores the left branch entirely before moving to the right branch. This exactly matches the order indicated in the Step 2 Pre-order diagram.

```cpp
// Post order
template <typename T>
void BinarySearchTree<T>::postOrder() {
    // Traverse left subtree
    if (left != nullptr) {
        left->postOrder();
    }

    // Traverse right subtree
    if (right != nullptr) {
        right->postOrder();
    }


    std::cout << key << " ";
}
```

```
PostOrder:
1 0 5 4 18 9 3 2
```

Observation: The output starts with the deepest leaf nodes on the left (1), processes the right subtree (5, 4, 18, 9, 3), and finishes with the root (2). This is completely different from the Pre-order output and exactly matches the order indicated in the Step 2 Post-order diagram.

Conclusion/Learnings/Analysis	Through this hands-on activity, we concluded that choosing between different Binary Search Tree traversal methods depends on the structure of the tree, the order in which nodes need to be accessed, and the specific goal of the operation itself. We discovered that Pre-order Traversal processes the current node first by printing the key before making recursive calls to the left and right subtrees, which is why our output started with the root value 2. This specific ordering is highly effective for copying or serializing a tree because it preserves the

	hierarchical structure from the top down. Moreover, we found out that In-order Traversal utilizes a similar recursive structure but waits to print the current node's key until after it has completely traversed the left subtree. This guarantees that the nodes are visited in ascending sorted order (0 to 18) due to the inherent properties of a Binary Search Tree, making it the most efficient method for data retrieval. In contrast, Post-order Traversal achieves the opposite effect by traversing both the left and right subtrees before finally processing and printing the root node last. By analyzing the outputs of these recursive functions, we discovered that Post-order is specifically advantageous for memory management tasks, such as deleting or freeing a tree, because it ensures child nodes are removed before their parent nodes. In our improvement that needs to be done, we want to improve our ability to visualize the tree structure before writing the traversal code because it is needed for understanding how the nodes connect because there are times that we mix up the left and right branches and end up predicting the wrong output order. We notice this mostly when we write it on a paper like the last quiz because we draw it using paper and pen only, so we cannot rely on the compiler to check our mistakes. We need to improve this skill to apply it to future coding activities efficiently.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.